

ODOO DEVELOPER PROGRAM

ORM Methods

Odoo 17 Development Tutorials

Ahmed Muhumed

July 25, 2026

OUTLINE

Introduction to ORM Methods

Create() ORM Method

write() ORM Method

unlink() ORM Method

copy() ORM Method

exists() ORM Method

search() ORM Method

read() ORM Method

search_read() ORM Method

search_count() ORM Method

name_create() ORM Method

default_get() ORM Method

name_search() ORM Method

get_view() ORM Method

ensure_one() ORM Method

filtered() ORM Method

mapped() ORM Method

sorted() ORM Method

grouped() ORM Method

fields_get() ORM Method

get_metadata() ORM Method

Introduction to ORM Methods

Introduction to ORM Methods

WHAT IS ORM?

- ▶ The ORM stands for **Object Relational Mapping**.
- ▶ It is a programming technique that allows developers to interact with a database using Python objects and methods instead of writing SQL queries **directly**.
- ▶ In Odoo, the ORM acts as a bridge between the Python application and the PostgreSQL database.
- ▶ Instead of writing:

```
1 INSERT INTO student_student (name, age) VALUES ('Ahmed', 22);
```

- ▶ we simply write:

```
1 self.env['student.student'].create({  
2     'name': 'Ahmed',  
3     'age': 22,  
4 })
```

WHAT DOES “OBJECT RELATIONAL MAPPING” MEAN?

► The term consists of three parts.

1. **Object** – an object is an instance of a Python class.

```
1 student = self.env['student.student'].browse(1)
```

Here, student is a Python object representing a student.

2. **Relational** – a relational database stores data in tables that are connected using relationships.

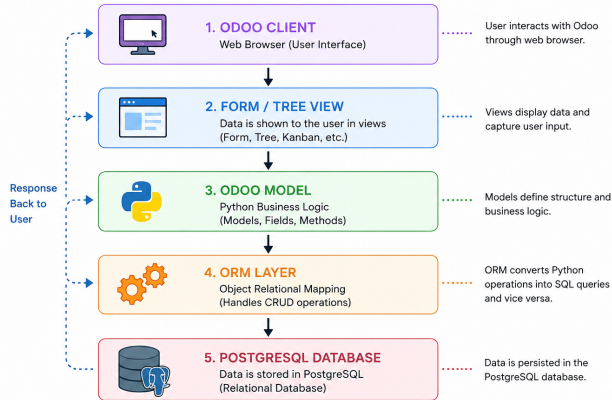
Students ↔ Classes (e.g. Mohamed ↔ Grade 12)

3. **Mapping** – mapping means connecting two different worlds.

Python Object ⇒ Odoo ORM ⇒ PostgreSQL Table

ORM ARCHITECTURE

ORM Architecture in Odoo



WHY IS ORM IMPORTANT?

- ▶ Without an ORM, developers would need to:
 - ☐ Write SQL for every operation.
 - ☐ Manage database connections manually.
 - ☐ Convert database rows into Python objects.
 - ☐ Prevent SQL injection.
- ▶ Although powerful, the ORM has limitations.
 - ☐ Less control over complex SQL queries.
 - ☐ Very large reports may require raw SQL.
 - ☐ Advanced database tuning sometimes needs direct SQL.

COMMON ORM METHODS

- ▶ `create`
- ▶ `write`
- ▶ `unlink`
- ▶ `copy`
- ▶ `exists`
- ▶ `search`
- ▶ `read`
- ▶ `search_read`
- ▶ `search_count`
- ▶ `name_create`
- ▶ `default_get`
- ▶ `name_search`
- ▶ `get_view()`
- ▶ `ensure_one()`
- ▶ `filtered()`
- ▶ `mapped()`
- ▶ `sorted()`
- ▶ `grouped()`
- ▶ `fields_get()`
- ▶ `get_metadata`

Create() **ORM Method**

Create() ORM Method

CREATE() METHOD

- ▶ The `create()` method is an ORM (Object Relational Mapping) method used to create one or more new records in an Odoo model.
- ▶ The `create()` method returns a **recordset** containing the newly created record.

Method Syntax

The standard syntax of the `create()` method is:

```
1 @api.model
2 def create(self, vals):
3     return super().create(vals)
```

This method has four important parts:

1. `@api.model`
2. `create`
3. `self`
4. `vals`

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `@api.model`: Indicates that the method is executed at the model level. The `@api.model` decorator tells Odoo that the method belongs to the model, not to an existing record.
- ▶ `create`: The ORM method responsible for creating records.
- ▶ `self`: The model (`student.student`), not a specific student record. If we are using `student.student` model, `self` does not represent a student record. Instead, `self` represents the model `student.student`.

```
1 print(self)
```

Output is:

```
1 student.student()
```

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `vals`: A dictionary containing the values to be stored in the database.

```
1 name = fields.Char()  
2 birth_date = fields.Date()  
3 gender = fields.Selection(...)
```

```
1 vals = {  
2     'name': 'Ahmed Mohamed',  
3     'birth_date': date(2005, 5, 10),  
4     'gender': 'male',  
5 }
```

- ▶ Each key corresponds to a field name in your model, and
- ▶ Each value is the data to store.
- ▶ The ORM maps each key to its corresponding field.

WHAT DOES CREATE() RETURN?

- ▶ The create() method returns a recordset.

```
1 student = self.env['student.student'].create({  
2     'name': 'Ahmed',  
3 })
```

- ▶ Now, if we try to print the student using print(student), we are getting this:

```
1 student.student(15,)
```

@API.MODEL VS @API.MODEL_CREATE_MULTI

There are two common decorators for create().

1. @api.model: Used for creating a single record.

```
1 @api.model
2 def create(self, vals):
```

vals is a dictionary.

```
1 student = self.env['student.student'].create({
2     'name': 'Ahmed Mohamed',
3 })
```

2. @api.model_create_multi: Used for creating multiple records efficiently.

```
1 @api.model_create_multi
2 def create(self, vals_list):
```

- ▶ `vals_list` is a list of dictionaries, i.e.,

```
1 [
2     {'name': 'Ahmed'},
3     {'name': 'Amina'},
4 ]
```

Method Call

- ▶ A method call is when we execute the method.

```
1 student = self.env['student.student'].create({
2     'name': 'Ahmed Mohamed',
3     'email': 'ahmed@example.com',
4 })
```

- ▶ Here, `.create(...)` is the method call.

`write()` **ORM Method**

`write()` **ORM Method**

WRITE() METHOD

- ▶ The `write()` method is an ORM method used to **update** one or more existing records in an Odoo model.
- ▶ Unlike `create()`, `write()` works on an existing recordset.
- ▶ The `write()` method returns a boolean value: `True` when the write succeeds.

Method Syntax

The standard syntax of the `write()` method is:

```
1 def write(self, vals):  
2     return super().write(vals)
```

This method has three important parts:

1. `write`
2. `self`
3. `vals`

BREAKING DOWN THE METHOD SIGNATURE

- ▶ **write**: The ORM method responsible for updating records.
- ▶ **self**: A recordset of one or more records that will be updated. If we browse student ID 15, **self** represents that student record.

```
1 student = self.env['student.student'].browse(15)
2 print(student)
```

Output is:

```
1 student.student(15,)
```

- ▶ If **self** contains many records, **write()** updates **all** of them with the same values.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `vals`: A dictionary containing the field values to update.

```
1 vals = {  
2     'name': 'Ahmed Muhumed',  
3     'email': 'ahmed@example.com',  
4     'age': 23,  
5 }
```

- ▶ Each key is a field name on the model.
- ▶ Each value is the new data to store.
- ▶ Fields that are not listed in `vals` stay unchanged.
- ▶ The ORM maps each key to its corresponding database column.

WHAT DOES WRITE() RETURN?

- ▶ The write() method returns True on success.

```
1 student = self.env['student.student'].browse(15)
2 result = student.write({
3     'name': 'Ahmed Muhumed',
4 })
5 print(result)
```

Output is:

```
1 True
```

- ▶ The record itself is updated in the database.
- ▶ You can continue using the same recordset after writing.

UPDATING ONE RECORD VS MANY RECORDS

1. One record:

```
1 student = self.env['student.student'].browse(15)
2 student.write({'age': 23})
```

2. Many records (same values for all):

```
1 students = self.env['student.student'].browse([15, 16, 17])
2 students.write({'active': True})
```

- ▶ One `write()` call on a multi-recordset is more efficient than looping and writing each record separately.
- ▶ Inverse and computed fields may also be recomputed after `write()`.

METHOD CALL AND OVERRIDE PATTERN

Method Call

- ▶ A method call is when we execute the method.

```
1 student.write({
2     'name': 'Ahmed Muhumed',
3     'email': 'ahmed@example.com',
4 })
```

- ▶ Here, `.write(...)` is the method call.

Common Override Pattern

```
1 def write(self, vals):
2     # custom logic before saving
3     if 'email' in vals:
4         vals['email'] = vals['email'].lower()
5     return super().write(vals)
```

`unlink()` **ORM Method**

`unlink()` **ORM Method**

UNLINK() METHOD

- ▶ The `unlink()` method is an ORM method used to **delete** one or more existing records from an Odoo model.
- ▶ In database terms, `unlink()` is similar to SQL `DELETE`.
- ▶ The method returns a boolean value: `True` when deletion succeeds.

Method Syntax

The standard syntax of the `unlink()` method is:

```
1 def unlink(self):  
2     return super().unlink()
```

This method has two important parts:

1. `unlink`
2. `self`

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `unlink`: The ORM method responsible for deleting records.
- ▶ `self`: A recordset of one or more records that will be deleted.

```
1 student = self.env['student.student'].browse(15)
2 print(student)
```

Output is:

```
1 student.student(15,)
```

- ▶ `unlink()` does **not** take a `vals` dictionary.
- ▶ It only needs the recordset that should be removed.

WHAT DOES UNLINK() RETURN?

- ▶ The `unlink()` method returns `True` on success.

```
1 student = self.env['student.student'].browse(15)
2 result = student.unlink()
3 print(result)
```

Output is:

```
1 True
```

- ▶ After `unlink()`, the record no longer exists in the database.
- ▶ Related one2many / many2many cleanup is handled by the ORM according to the field `ondelete` behavior.

DELETING ONE RECORD VS MANY RECORDS

1. One record:

```
1 student = self.env['student.student'].browse(15)
2 student.unlink()
```

2. Many records:

```
1 students = self.env['student.student'].search([
2     ('active', '=', False),
3 ])
4 students.unlink()
```

- ▶ Prefer one `unlink()` on a recordset over deleting records in a Python loop.
- ▶ Access rights and record rules still apply before deletion.

METHOD CALL AND OVERRIDE PATTERN

Method Call

```
1 student.unlink()
```

- ▶ Here, `.unlink()` is the method call.

Common Override Pattern

```
1 def unlink(self):  
2     for record in self:  
3         if record.state == 'done':  
4             raise UserError("Done students cannot be deleted.")  
5     return super().unlink()
```

- ▶ Overrides are often used to block unsafe deletions.

IMPORTANT NOTES ABOUT `UNLINK()`

- ▶ Deletion is permanent for normal models (unless you use `archive/active`).
- ▶ Many business models prefer **archiving** instead of deleting:

```
1 student.write({'active': False}) # soft delete / archive
```

- ▶ Foreign-key constraints may raise errors if other records still depend on the record you are deleting.
- ▶ Always check business rules before calling `unlink()` in production code.

`copy()` **ORM Method**

`copy()` **ORM Method**

COPY() METHOD

- ▶ The `copy()` method is an ORM method used to **duplicate** an existing record.
- ▶ It creates a new record by copying field values from the current record.
- ▶ The method returns a recordset containing the newly created (duplicated) record.

Method Syntax

The standard syntax of the `copy()` method is:

```
1 def copy(self, default=None):  
2     return super().copy(default)
```

This method has three important parts:

1. `copy`
2. `self`
3. `default`

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `copy`: The ORM method responsible for duplicating records.
- ▶ `self`: The source recordset to copy from. In normal usage, `self` is usually **one** record.
- ▶ `default`: An optional dictionary of values that override the copied values.

```
1 default = {  
2     'name': 'Ahmed Mohamed (Copy)',  
3 }
```

- ▶ Keys in `default` replace the values that would otherwise be copied.
- ▶ Fields not listed in `default` are copied from the original record (unless marked `copy=False`).

WHAT DOES `COPY()` RETURN?

- ▶ The `copy()` method returns a new recordset.

```
1 student = self.env['student.student'].browse(15)
2 new_student = student.copy()
3 print(new_student)
```

Output is (example):

```
1 student.student(28,)
```

- ▶ A new database row is created.
- ▶ The original record remains unchanged.

USING THE DEFAULT ARGUMENT

- ▶ You can change some values while copying.

```
1 student = self.env['student.student'].browse(15)
2 new_student = student.copy({
3     'name': 'Amina Mohamed',
4     'email': 'amina@example.com',
5 })
```

- ▶ Everything else is copied from student 15.
- ▶ This is useful for “Duplicate” buttons in the UI.

WHICH FIELDS ARE COPIED?

- ▶ By default, most stored fields are copied.
- ▶ A field can opt out of copying:

```
1 email = fields.Char(copy=False)  
2 registration_code = fields.Char(copy=False)
```

- ▶ Fields with `copy=False` become empty / default on the new record.
- ▶ One2many lines may also be copied depending on the field definition.
- ▶ Unique constraints may force you to override values through default.

METHOD CALL AND OVERRIDE PATTERN

Method Call

```
1 new_student = student.copy({'name': 'Ahmed (Copy)'})
```

Common Override Pattern

```
1 def copy(self, default=None):  
2     default = dict(default or {})  
3     default.setdefault('name', f"{self.name} (Copy)")  
4     return super().copy(default)
```

- Overrides are useful when duplicated names must stay unique.

`exists()` **ORM Method**

`exists()` **ORM Method**

EXISTS() METHOD

- ▶ The `exists()` method checks whether the records in a recordset still exist in the database.
- ▶ It is useful when a recordset may contain deleted or invalid IDs.
- ▶ The method returns a recordset containing only the records that still exist.

Method Syntax

The standard syntax of the `exists()` method is:

```
1 def exists(self):  
2     return super().exists()
```

This method has two important parts:

1. `exists`
2. `self`

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `exists`: The ORM method that filters a recordset to real DB rows.
- ▶ `self`: The recordset you want to validate.

```
1 students = self.env['student.student'].browse([15, 16, 9999])  
2 print(students)
```

Output is:

```
1 student.student(15, 16, 9999)
```

- ▶ `browse()` does not check the database immediately.
- ▶ ID 9999 may not exist, but it is still inside the recordset.

WHAT DOES EXISTS() RETURN?

- ▶ exists() returns a recordset of only the records that exist.

```
1 students = self.env['student.student'].browse([15, 16, 9999])
2 real_students = students.exists()
3 print(real_students)
```

Output is (example):

```
1 student.student(15, 16)
```

- ▶ ID 9999 was removed because it is not in the database.

CHECKING A SINGLE RECORD

- ▶ You can also test one record in an if statement.

```
1 student = self.env['student.student'].browse(15)
2 if student.exists():
3     print("Student is still in the database")
4 else:
5     print("Student was deleted")
```

- ▶ An empty recordset is falsy in boolean context.
- ▶ This pattern avoids crashing when a record was already unlinked.

WHEN SHOULD YOU USE EXISTS()?

- ▶ After long-running operations where records may have been deleted.
- ▶ When you received IDs from the UI, RPC, or another model.
- ▶ Before writing or unlinking a recordset that might be stale.

```
1 records = self.env['student.student'].browse(ids).exists()  
2 records.write({'active': True})
```

- ▶ `exists()` is a safety method: it does not create or update data.

Method Call

```
1 real_students = students.exists()
```

- ▶ Here, `.exists()` is the method call.
- ▶ No dictionary argument is required.
- ▶ The return value is always a recordset (possibly empty).

`search()` **ORM Method**

`search()` **ORM Method**

- ▶ The `search()` method is an ORM method used to **find records** that match a domain (filter condition).
- ▶ It is one of the most used ORM methods in Odoo.
- ▶ The method returns a recordset of matching records.

Method Syntax

The standard syntax of the `search()` method is:

```
1 @api.model
2 def search(self, domain, offset=0, limit=None, order=None):
3     return super().search(domain, offset=offset, limit=limit, order=order)
```

Important parts: `domain`, `offset`, `limit`, `order`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `@api.model`: Search runs at model level (not on one existing record).
- ▶ `self`: Represents the model, e.g. `student.student()`.
- ▶ `domain`: A list of conditions used to filter records.

```
1 domain = [('age', '>=', 18), ('gender', '=', 'male')]
```

- ▶ Each condition is a tuple: (field_name, operator, value).
- ▶ Multiple conditions are combined with AND by default.

BREAKING DOWN DOMAIN OPERATORS

Common operators used inside a domain:

- ▶ = equal
- ▶ != not equal
- ▶ > >= < <= comparisons
- ▶ like / ilike text search (ilike is case-insensitive)
- ▶ in / not in membership
- ▶ child_of hierarchical search

```
1 [('name', 'ilike', 'ahmed')]  
2 [('id', 'in', [15, 16, 17])]  
3 ['|', ('gender', '=', 'male'), ('age', '<', 18)]
```

- ▶ '|' means OR, '&' means AND, '!' means NOT.

OFFSET, LIMIT, AND ORDER

- ▶ offset: Skip the first N records (used for pagination).
- ▶ limit: Return at most N records.
- ▶ order: SQL-like ordering string.

```
1 students = self.env['student.student'].search(  
2     [('active', '=', True)],  
3     offset=10,  
4     limit=5,  
5     order='name asc, id desc',  
6 )
```

- ▶ This returns active students 11–15 when sorted by name.

WHAT DOES SEARCH() RETURN?

- ▶ The `search()` method returns a recordset.

```
1 students = self.env['student.student'].search([
2     ('name', 'ilike', 'ahmed'),
3 ])
4 print(students)
```

Output is (example):

```
1 student.student(15, 22, 31)
```

- ▶ If nothing matches, it returns an empty recordset: `student.student()`.
- ▶ An empty recordset is falsy in an if condition.

METHOD CALL AND PRACTICAL EXAMPLE

Method Call

```
1 students = self.env['student.student'].search([  
2     ('age', '>=', 18),  
3     ('active', '=', True),  
4 ], limit=20, order='age desc')
```

- ▶ Here, `.search(...)` is the method call.
- ▶ Always prefer domains over building raw SQL for normal queries.
- ▶ Record rules and access rights are applied automatically.

`read()` **ORM Method**

`read()` **ORM Method**

- ▶ The read() method is an ORM method used to **fetch field values** from a recordset as plain Python dictionaries.
- ▶ It is commonly used by the web client and RPC calls.
- ▶ The method returns a **list of dictionaries**, not a recordset.

Method Syntax

The standard syntax of the read() method is:

```
1 def read(self, fields=None, load='_classic_read'):  
2     return super().read(fields=fields, load=load)
```

Important parts: self, fields, load.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `self`: The recordset whose values you want to read.
- ▶ `fields`: An optional list of field names to fetch.

```
1 fields = ['name', 'age', 'email']
```

- ▶ If `fields` is omitted, Odoo reads all readable fields.
- ▶ Reading only needed fields is faster and cleaner.
- ▶ `load`: Controls how relational values are returned (classic read vs plain IDs).

WHAT DOES READ() RETURN?

- ▶ read() returns a list of dictionaries.

```
1 student = self.env['student.student'].browse(15)
2 data = student.read(['name', 'age'])
3 print(data)
```

Output is (example):

```
1 [{ 'id': 15, 'name': 'Ahmed', 'age': 22}]
```

- ▶ Each dictionary represents one record.
- ▶ The key id is always included.

READING MANY RECORDS

```
1 students = self.env['student.student'].browse([15, 16])
2 data = students.read(['name', 'email'])
```

Output is (example):

```
1 [
2     {'id': 15, 'name': 'Ahmed', 'email': 'ahmed@example.com'},
3     {'id': 16, 'name': 'Amina', 'email': 'amina@example.com'},
4 ]
```

- ▶ Useful when you need serializable data (JSON / RPC / API).
- ▶ For normal business logic inside Odoo, prefer recordset access: `student.name`.

1. Recordset access (preferred in Python business code):

```
1 name = student.name  
2 age = student.age
```

2. read() (preferred for RPC / exporting dict data):

```
1 data = student.read(['name', 'age'])[0]
```

- ▶ Recordset access keeps ORM features (prefetch, cache, relational browsing).
- ▶ read() gives simple data structures.

Method Call

```
1 data = self.env['student.student'].browse(15).read([
2     'name',
3     'birth_date',
4     'gender',
5 ])
```

- ▶ Here, `.read(...)` is the method call.
- ▶ Return type reminder: **list of dicts**, not a recordset.

`search_read()` **ORM Method**

`search_read()` **ORM Method**

SEARCH_READ() METHOD

- ▶ The `search_read()` method combines `search()` and `read()` in one call.
- ▶ It finds records with a domain, then returns their field values as dictionaries.
- ▶ It is heavily used by the Odoo web client.

Method Syntax

```
1 @api.model
2 def search_read(self, domain=None, fields=None,
3                 offset=0, limit=None, order=None):
4     return super().search_read(domain=domain, fields=fields,
5                               offset=offset, limit=limit, order=order)
```

BREAKING DOWN THE METHOD SIGNATURE

- ▶ domain: Filter conditions (same format as `search()`).
- ▶ fields: List of field names to return.
- ▶ offset: Number of records to skip.
- ▶ limit: Maximum number of records.
- ▶ order: Sort expression.

```
1 domain = [('active', '=', True)]  
2 fields = ['name', 'age', 'email']
```

WHAT DOES SEARCH_READ() RETURN?

► It returns a **list of dictionaries**.

```
1 data = self.env['student.student'].search_read(  
2     [('age', '>=', 18)],  
3     ['name', 'age'],  
4     limit=2,  
5     order='age desc',  
6 )  
7 print(data)
```

Output is (example):

```
1 [  
2     {'id': 22, 'name': 'Amina', 'age': 25},  
3     {'id': 15, 'name': 'Ahmed', 'age': 22},  
4 ]
```

SEARCH_READ() VS SEARCH() + READ()

Equivalent ideas:

```
1 # Two calls
2 records = self.env['student.student'].search(domain, limit=10)
3 data = records.read(['name', 'age'])
4
5 # One call
6 data = self.env['student.student'].search_read(
7     domain, ['name', 'age'], limit=10,
8 )
```

- ▶ `search_read()` is convenient and often more efficient for RPC.
- ▶ Inside server business logic, `search()` + recordset access is usually clearer.

WHEN TO USE SEARCH_READ()

- ▶ Building API responses.
- ▶ Returning table/list data to the UI.
- ▶ Exporting selected fields without keeping a recordset around.

```
1 rows = self.env['student.student'].search_read(  
2     [('gender', '=', 'female')],  
3     ['name', 'email', 'birth_date'],  
4     order='name asc',  
5 )
```

Method Call

```
1 rows = self.env['student.student'].search_read(  
2     [('active', '=', True)],  
3     fields=['name', 'age'],  
4     offset=0,  
5     limit=50,  
6     order='id desc',  
7 )
```

- ▶ Here, `.search_read(...)` is the method call.
- ▶ Remember: return type is a list of dicts.

`search_count()` **ORM Method**

`search_count()` **ORM Method**

- ▶ The `search_count()` method returns the **number of records** that match a domain.
- ▶ It is more efficient than searching all records and then using `len()`.
- ▶ The method returns an integer.

Method Syntax

```
1 @api.model
2 def search_count(self, domain, limit=None):
3     return super().search_count(domain, limit=limit)
```

Important parts: domain, optional limit.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `@api.model`: Executed at the model level.
- ▶ `self`: The model, not a specific record.
- ▶ `domain`: Same domain language used by `search()`.
- ▶ `limit`: Optional maximum count (less commonly used).

```
1 domain = [('active', '=', True), ('age', '>=', 18)]
```

WHAT DOES SEARCH_COUNT() RETURN?

- ▶ It returns an **integer**.

```
1 total = self.env['student.student'].search_count([
2     ('gender', '=', 'male'),
3 ])
4 print(total)
```

Output is (example):

```
1 12
```

- ▶ No recordset is created.
- ▶ Only the count is computed.

WHY NOT USE LEN(SEARCH(...))?

Less efficient pattern:

```
1 total = len(self.env['student.student'].search([
2     ('active', '=', True),
3 ]))
```

Better pattern:

```
1 total = self.env['student.student'].search_count([
2     ('active', '=', True),
3 ])
```

- ▶ `search_count()` avoids loading every matching record into Python.
- ▶ Important when tables are large.

PRACTICAL EXAMPLES

```
1 adults = self.env['student.student'].search_count([
2     ('age', '>=', 18),
3 ])
4
5 archived = self.env['student.student'].search_count([
6     ('active', '=', False),
7 ])
8
9 has_email = self.env['student.student'].search_count([
10    ('email', '!=', False),
11 ])
```

- Useful for dashboards, KPIs, and validation checks.

Method Call

```
1 count = self.env['student.student'].search_count([  
2     ('name', 'ilike', 'ahmed'),  
3 ])
```

- ▶ Here, `.search_count(...)` is the method call.
- ▶ Return type reminder: **int**.

`name_create()` **ORM Method**

`name_create()` **ORM Method**

NAME_CREATE() METHOD

- ▶ The `name_create()` method creates a record using only a display name.
- ▶ It is used by many2one fields when the user types a new value and quick-creates it.
- ▶ Internally, it usually calls `create()` with the `name` field.

Method Syntax

```
1 @api.model
2 def name_create(self, name):
3     return super().name_create(name)
```

Important parts: `@api.model`, `name`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `@api.model`: Runs at the model level.
- ▶ `self`: The model on which the new record is created.
- ▶ `name`: A string that becomes the record's name / display name.

```
1 name = "Ahmed Mohamed"
```

- ▶ No full `vals` dictionary is required by the caller.
- ▶ Extra required fields must be handled by defaults or an override.

WHAT DOES NAME_CREATE() RETURN?

- ▶ It returns a tuple: (id, display_name).

```
1 result = self.env['student.student'].name_create('Ahmed Mohamed')
2 print(result)
```

Output is (example):

```
1 (29, 'Ahmed Mohamed')
```

- ▶ First value: database ID of the new record.
- ▶ Second value: display name shown in the UI.

HOW THE UI USES NAME_CREATE()

- ▶ In a many2one field, the user types a name that does not exist.
- ▶ Odoo offers “Create ...” / quick create.
- ▶ Selecting that option calls `name_create()`.

```
1 # Conceptual flow
2 # typed value: "Grade 12"
3 self.env['school.class'].name_create('Grade 12')
```

- ▶ This is why many2one quick-create only needs a name.

OVERRIDE PATTERN

When a model has more required fields than name:

```
1 @api.model
2 def name_create(self, name):
3     record = self.create({
4         'name': name,
5         'active': True,
6         'gender': 'other',
7     })
8     return record.id, record.display_name
```

- Override `name_create()` when quick-create must fill extra values.

Method Call

```
1 record_id, display_name = self.env['student.student'].name_create(  
2     'Ahmed Mohamed'  
3 )
```

- ▶ Here, `.name_create(...)` is the method call.
- ▶ Return type reminder: **tuple (id, display_name)**.

`default_get()` **ORM Method**

default_get() **ORM Method**

DEFAULT_GET() METHOD

- ▶ The `default_get()` method returns the **default values** used when creating a new record in the UI or through the ORM.
- ▶ It collects defaults from field definitions, context, and company settings.
- ▶ The method returns a dictionary of field values.

Method Syntax

```
1 @api.model
2 def default_get(self, fields_list):
3     return super().default_get(fields_list)
```

Important parts: `fields_list`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `@api.model`: Model-level method.
- ▶ `self`: The model requesting defaults.
- ▶ `fields_list`: List of field names for which defaults are needed.

```
1 fields_list = ['name', 'gender', 'active', 'company_id']
```

- ▶ The UI calls this when you click **New** on a form.

WHAT DOES DEFAULT_GET() RETURN?

- ▶ It returns a dictionary of default values.

```
1 defaults = self.env['student.student'].default_get([
2     'name', 'gender', 'active',
3 ])
4 print(defaults)
```

Output is (example):

```
1 {'active': True, 'gender': 'male'}
```

- ▶ Only fields that have defaults appear in the dictionary.
- ▶ Missing keys mean “no default”.

WHERE DEFAULTS COME FROM

1. Field-level default:

```
1 active = fields.Boolean(default=True)
2 gender = fields.Selection(..., default='male')
```

2. Context default:

```
1 self.env['student.student'].with_context(
2     default_gender='female'
3 ).default_get(['gender'])
```

3. Custom logic inside an overridden default_get().

OVERRIDE PATTERN

```
1 @api.model
2 def default_get(self, fields_list):
3     defaults = super().default_get(fields_list)
4     if 'name' in fields_list:
5         defaults['name'] = 'New Student'
6     if 'active' in fields_list:
7         defaults['active'] = True
8     return defaults
```

- ▶ Always call `super()` first, then adjust the dictionary.
- ▶ Only set keys that were requested in `fields_list`.

Method Call

```
1 vals = self.env['student.student'].default_get([  
2     'name', 'birth_date', 'gender', 'active',  
3 ])
```

- ▶ Here, `.default_get(...)` is the method call.
- ▶ Return type reminder: **dict**.

`name_search()` **ORM Method**

`name_search()` **ORM Method**

NAME_SEARCH() METHOD

- ▶ The `name_search()` method searches records by name / display name.
- ▶ It powers many2one dropdown search-as-you-type behavior.
- ▶ It returns a list of (id, display_name) tuples.

Method Syntax

```
1 @api.model
2 def name_search(self, name='', args=None,
3                 operator='ilike', limit=100):
4     return super().name_search(name=name, args=args,
5                               operator=operator, limit=limit)
```

BREAKING DOWN THE METHOD SIGNATURE

- ▶ name: Text typed by the user.
- ▶ args: Extra domain restrictions.
- ▶ operator: Comparison operator, usually *ilike*.
- ▶ limit: Maximum number of suggestions.

```
1 name = 'ahm'
2 args = [('active', '=', True)]
3 operator = 'ilike'
4 limit = 8
```

WHAT DOES NAME_SEARCH() RETURN?

- It returns a list of tuples: (id, display_name).

```
1 result = self.env['student.student'].name_search('ahm')
2 print(result)
```

Output is (example):

```
1 [
2     (15, 'Ahmed Mohamed'),
3     (22, 'Ahmed Ali'),
4 ]
```

- Perfect format for many2one selection lists.

HOW MANY2ONE USES NAME_SEARCH()

- ▶ User opens a many2one field and types text.
- ▶ Odoo calls `name_search()` with that text.
- ▶ Matching records appear in the dropdown.

```
1 self.env['student.student'].name_search(  
2     name='amina',  
3     args=[('active', '=', True)],  
4     operator='ilike',  
5     limit=10,  
6 )
```

SEARCHING EXTRA FIELDS

By default, name search focuses on the name / display name. You can extend it:

```
1 @api.model
2 def name_search(self, name='', args=None,
3                 operator='ilike', limit=100):
4     args = args or []
5     domain = ['|',
6              ('name', operator, name),
7              ('email', operator, name)]
8     records = self.search(domain + args, limit=limit)
9     return [(rec.id, rec.display_name) for rec in records]
```

► Now users can find a student by name or email.

Method Call

```
1 matches = self.env['student.student'].name_search(  
2     name='ahmed',  
3     args=[('gender', '=', 'male')],  
4     operator='ilike',  
5     limit=5,  
6 )
```

- ▶ Here, `.name_search(...)` is the method call.
- ▶ Return type reminder: **list of (id, display_name)**.

`get_view()` **ORM Method**

`get_view()` **ORM Method**

GET_VIEW() METHOD

- ▶ The `get_view()` method returns the architecture and metadata of a view for a model (form, list/tree, kanban, search, ...).
- ▶ The web client uses it to know how to render a screen.
- ▶ In Odoo 17, `get_view()` replaces older multi-view helpers such as `fields_view_get()` in many flows.

Method Syntax

```
1 @api.model
2 def get_view(self, view_id=None, view_type='form', **options):
3     return super().get_view(view_id=view_id, view_type=view_type, **options)
```

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `view_id`: Optional specific view XML ID / database ID.
- ▶ `view_type`: Type of view to load: `'form'`, `'list'`, `'kanban'`, `'search'`, ...
- ▶ `options`: Extra options used by the client / ORM.

```
1 view_type = 'form'  
2 view_id = None # let Odoo choose the default form view
```

WHAT DOES GET_VIEW() RETURN?

- ▶ It returns a dictionary of view information.

```
1 info = self.env['student.student'].get_view(view_type='form')
2 print(info.keys())
```

Common keys include:

- ▶ id – view record ID
- ▶ name – view name
- ▶ type – view type
- ▶ arch – the XML architecture
- ▶ models – field/model metadata needed by the view

PRACTICAL EXAMPLE

```
1 form_view = self.env['student.student'].get_view(view_type='form')
2 list_view = self.env['student.student'].get_view(view_type='list')
3
4 print(form_view['type'])      # form
5 print(list_view['type'])     # list
6 print(form_view['arch'])     # XML architecture string/element
```

- ▶ Developers rarely call this in business methods.
- ▶ It is essential for UI, studio-like tools, and technical debugging.

WHEN IS GET_VIEW() USEFUL?

- ▶ Understanding which view Odoo will open for a model.
- ▶ Building custom clients that render Odoo views.
- ▶ Debugging inherited / extended view architecture.
- ▶ Inspecting the final combined XML after view inheritance.

```
1 info = self.env['student.student'].get_view(view_type='search')
```

Method Call

```
1 view_info = self.env['student.student'].get_view(  
2     view_id=None,  
3     view_type='form',  
4 )
```

- ▶ Here, `.get_view(...)` is the method call.
- ▶ Return type reminder: **dict** of view metadata.

`ensure_one()` **ORM Method**

`ensure_one()` **ORM Method**

- ▶ The `ensure_one()` method verifies that a recordset contains **exactly one** record.
- ▶ If the recordset is empty or contains multiple records, it raises an error.
- ▶ It returns the same recordset when the check passes.

Method Syntax

```
1 def ensure_one(self):  
2     return super().ensure_one()
```

Important parts: `ensure_one`, `self`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `ensure_one`: Safety check method for singleton recordsets.
- ▶ `self`: The recordset being validated.
- ▶ No extra arguments are required.

```
1 student = self.env['student.student'].browse(15)
2 student.ensure_one()
```

- ▶ This succeeds because there is exactly one record.

WHAT DOES ENSURE_ONE() RETURN?

- ▶ On success, it returns the same recordset.

```
1 student = self.env['student.student'].browse(15)
2 result = student.ensure_one()
3 print(result)
```

Output is:

```
1 student.student(15,)
```

- ▶ On failure, it raises ValueError.

WHEN IT FAILS

Empty recordset:

```
1 students = self.env['student.student'].browse([])
2 students.ensure_one() # ValueError
```

Multiple records:

```
1 students = self.env['student.student'].browse([15, 16])
2 students.ensure_one() # ValueError
```

► Error message idea: “Expected singleton”.

WHY IS ENSURE_ONE() IMPORTANT?

- ▶ Many methods are written for one record only (send email, print report, open wizard).
- ▶ Accessing `record.name` on a multi-recordset can raise singleton errors later.
- ▶ Calling `ensure_one()` makes the expectation explicit and fails early.

```
1 def action_confirm(self):  
2     self.ensure_one()  
3     self.state = 'confirmed'  
4     return True
```

Method Call

```
1 self.ensure_one()
```

- ▶ Here, `.ensure_one()` is the method call.
- ▶ Use it at the start of button actions and record-specific business methods.

`filtered()` ORM Method

`filtered()` **ORM Method**

- ▶ The `filtered()` method filters an existing recordset in Python.
- ▶ Unlike `search()`, it does **not** run a new SQL query by domain.
- ▶ It returns a new recordset containing only records that match a condition.

Method Syntax

```
1 def filtered(self, func):  
2     return super().filtered(func)
```

Important parts: `self`, `func`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `self`: The starting recordset.
- ▶ `func`: Either
 - ❑ a function / lambda that returns True/False, or
 - ❑ a field name string (truthy check).

```
1 students.filtered(lambda s: s.age >= 18)
2 students.filtered('email')    # keep records with a truthy email
```

WHAT DOES FILTERED() RETURN?

► It returns a recordset.

```
1 students = self.env['student.student'].search([])
2 adults = students.filtered(lambda s: s.age >= 18)
3 print(adults)
```

Output is (example):

```
1 student.student(15, 22)
```

FILTERED WITH A LAMBDA

```
1 students = self.env['student.student'].browse([15, 16, 17])
2
3 males = students.filtered(lambda s: s.gender == 'male')
4 with_email = students.filtered(lambda s: bool(s.email))
5 active_adults = students.filtered(
6     lambda s: s.active and s.age >= 18
7 )
```

- ▶ The lambda receives one record at a time.
- ▶ Return True to keep the record.

FILTERED() VS SEARCH()

- ▶ Use `search()` when filtering from the whole table with a domain (SQL).
- ▶ Use `filtered()` when you already have a recordset and need a local filter.

```
1 # From database
2 adults = self.env['student.student'].search([('age', '>=', 18)])
3
4 # From an existing recordset
5 adults = students.filtered(lambda s: s.age >= 18)
```

Method Call

```
1 adults = students.filtered(lambda student: student.age >= 18)
```

- ▶ Here, `.filtered(...)` is the method call.
- ▶ Return type reminder: **recordset**.

`mapped()` **ORM Method**

`mapped()` **ORM Method**

- ▶ The `mapped()` method extracts values or related records from a recordset.
- ▶ It is used to map each record to a field, expression, or function result.
- ▶ The return type depends on what you map to.

Method Syntax

```
1 def mapped(self, func):  
2     return super().mapped(func)
```

Important parts: `self`, `func`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `self`: The source recordset.
- ▶ `func`: A field path string, or a function / lambda.

```
1 students.mapped('name')
2 students.mapped('classroom_id')
3 students.mapped('classroom_id.name')
4 students.mapped(lambda s: s.age * 2)
```

WHAT DOES MAPPED() RETURN?

Mapping a simple field returns a Python list:

```
1 names = students.mapped('name')
2 print(names)
```

```
1 ['Ahmed', 'Amina', 'Omar']
```

Mapping a relational field returns a recordset:

```
1 classes = students.mapped('classroom_id')
2 print(classes)
```

```
1 school.classroom(3, 5)
```

DOT-PATH MAPPING

You can follow relational paths:

```
1 class_names = students.mapped('classroom_id.name')  
2 teacher_names = students.mapped('classroom_id.teacher_id.name')
```

- ▶ Odoo walks the relationship for each record.
- ▶ Duplicate relational records are compressed in the resulting recordset.
- ▶ This is cleaner than writing manual loops.

MAPPING WITH A FUNCTION

```
1 labels = students.mapped(  
2     lambda s: f"{s.name} ({s.age})"  
3 )  
4 ages = students.mapped(lambda s: s.age)  
5 total_age = sum(students.mapped('age'))
```

- ▶ Function mapping always returns a list.
- ▶ Very useful for reports, totals, and display labels.

Method Call

```
1 emails = students.mapped('email')  
2 classrooms = students.mapped('classroom_id')
```

- ▶ Here, `.mapped(...)` is the method call.
- ▶ Return type: **list** or **recordset**, depending on the mapped path.

`sorted()` **ORM Method**

`sorted()` **ORM Method**

SORTED() METHOD

- ▶ The `sorted()` method returns a recordset ordered in Python.
- ▶ It sorts an existing recordset; it does not query the database again.
- ▶ The method returns a new sorted recordset.

Method Syntax

```
1 def sorted(self, key=None, reverse=False):  
2     return super().sorted(key=key, reverse=reverse)
```

Important parts: `key`, `reverse`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `self`: The recordset to sort.
- ▶ `key`: A field name string, or a function that returns the sort value.
- ▶ `reverse`: False for ascending, True for descending.

```
1 students.sorted('name')  
2 students.sorted(key=lambda s: s.age, reverse=True)
```

WHAT DOES SORTED() RETURN?

- It returns a recordset in the new order.

```
1 students = self.env['student.student'].browse([17, 15, 16])
2 ordered = students.sorted('name')
3 print(ordered)
```

Output is (example):

```
1 student.student(15, 16, 17)
```

SORTING BY FIELD OR LAMBDA

```
1 by_name = students.sorted('name')
2 by_age_desc = students.sorted('age', reverse=True)
3 by_email = students.sorted(key=lambda s: s.email or '')
4 by_class_then_name = students.sorted(
5     key=lambda s: (s.classroom_id.name or '', s.name or ''))
6 )
```

- Use a lambda when you need multi-key or computed sorting.

SORTED() VS SEARCH(... ORDER=)

- ▶ `search(..., order='age desc')` sorts in SQL while fetching.
- ▶ `sorted()` sorts an already loaded recordset in Python.

```
1 # SQL ordering
2 students = self.env['student.student'].search([], order='age desc')
3
4 # Python ordering
5 students = students.sorted('age', reverse=True)
```

Method Call

```
1 ordered_students = students.sorted(key='age', reverse=True)
```

- ▶ Here, `.sorted(...)` is the method call.
- ▶ Return type reminder: **recordset**.

`grouped()` ORM Method

grouped() ORM Method

- ▶ The `grouped()` method groups an existing recordset by a field or key.
- ▶ It is available on recordsets in modern Odoo versions (including Odoo 17).
- ▶ It returns a dictionary mapping each group key to a recordset.

Method Syntax

```
1 def grouped(self, key):  
2     return super().grouped(key)
```

Important parts: `self`, `key`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `self`: The recordset to group.
- ▶ `key`: A field name string, or a function that returns the group key.

```
1 students.grouped('gender')
2 students.grouped('classroom_id')
3 students.grouped(lambda s: s.age // 10)  # age decade groups
```

WHAT DOES GROUPED() RETURN?

► It returns a dictionary.

```
1 students = self.env['student.student'].search([])
2 groups = students.grouped('gender')
3 print(groups)
```

Output is (example):

```
1 {
2     'male': student.student(15, 17),
3     'female': student.student(16, 22),
4 }
```

GROUPING BY A MANY2ONE FIELD

```
1 grouped_by_class = students.grouped('classroom_id')
2
3 for classroom, class_students in grouped_by_class.items():
4     print(classroom.name, len(class_students))
```

- ▶ Keys are relational records (or empty record / falsy key when empty).
- ▶ Values are recordsets of students in that group.

PRACTICAL USE CASES

- ▶ Build totals per group without writing SQL GROUP BY.
- ▶ Split records before batch processing.
- ▶ Prepare data for reports and dashboards.

```
1 groups = students.grouped('gender')
2 male_count = len(groups.get('male', self.env['student.student']))
3 female_count = len(groups.get('female', self.env['student.student']))
```

Method Call

```
1 groups = students.grouped('classroom_id')
```

- ▶ Here, `.grouped(...)` is the method call.
- ▶ Return type reminder: **dict[key → recordset]**.

`fields_get()` **ORM Method**

`fields_get()` **ORM Method**

- ▶ The `fields_get()` method returns **metadata** about model fields.
- ▶ It describes field type, string/label, required, selection values, relation, etc.
- ▶ The web client uses it to understand how fields should behave in the UI.

Method Syntax

```
1 @api.model
2 def fields_get(self, allfields=None, attributes=None):
3     return super().fields_get(allfields=allfields, attributes=attributes)
```

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `allfields`: Optional list of field names to inspect. If omitted, metadata for all fields is returned.
- ▶ `attributes`: Optional list of attribute names to include (for example `string`, `type`, `required`).

```
1 self.env['student.student'].fields_get(['name', 'gender'])
2 self.env['student.student'].fields_get(
3     ['age'], attributes=['string', 'type', 'required']
4 )
```

WHAT DOES FIELDS_GET() RETURN?

- It returns a dictionary of dictionaries.

```
1 info = self.env['student.student'].fields_get(['name', 'gender'])
2 print(info)
```

Output is (example):

```
1 {
2   'name': {'type': 'char', 'string': 'Name', 'required': True, ...},
3   'gender': {
4     'type': 'selection',
5     'string': 'Gender',
6     'selection': [('male', 'Male'), ('female', 'Female')],
7     ...
8   },
9 }
```

USEFUL METADATA KEYS

Common keys you will see:

- ▶ `type` – field type (`char`, `many2one`, ...)
- ▶ `string` – field label
- ▶ `required` – whether the field is required
- ▶ `readonly` – whether the field is read-only
- ▶ `help` – help text
- ▶ `selection` – values for selection fields
- ▶ `relation` – comodel name for relational fields

PRACTICAL EXAMPLE

```
1 meta = self.env['student.student'].fields_get(  
2     ['name', 'classroom_id', 'gender'],  
3     attributes=['string', 'type', 'required', 'relation', 'selection'],  
4 )  
5  
6 print(meta['classroom_id']['type'])          # many2one  
7 print(meta['classroom_id']['relation'])      # school.classroom  
8 print(meta['gender']['selection'])
```

- Very useful for dynamic UI, generic exports, and technical debugging.

Method Call

```
1 fields_info = self.env['student.student'].fields_get(  
2     allfields=['name', 'age', 'email'],  
3     attributes=['string', 'type', 'required'],  
4 )
```

- ▶ Here, `.fields_get(...)` is the method call.
- ▶ Return type reminder: **dict**.

`get_metadata()` **ORM Method**

`get_metadata()` **ORM Method**

GET_METADATA() METHOD

- ▶ The `get_metadata()` method returns technical metadata about records.
- ▶ It includes information such as XML IDs, create/write audit fields, and whether the record is a noupdate data record.
- ▶ It is mostly used by technical tools and the UI debug / metadata dialogs.

Method Syntax

```
1 def get_metadata(self):  
2     return super().get_metadata()
```

Important parts: `get_metadata`, `self`.

BREAKING DOWN THE METHOD SIGNATURE

- ▶ `self`: The recordset to inspect.
- ▶ No values dictionary is required.
- ▶ Works on one or many records; returns one metadata dict per record.

```
1 student = self.env['student.student'].browse(15)
2 meta = student.get_metadata()
```

WHAT DOES GET_METADATA() RETURN?

► It returns a **list of dictionaries**.

```
1 meta = self.env['student.student'].browse(15).get_metadata()
2 print(meta)
```

Output is (example):

```
1 [{
2     'id': 15,
3     'xmlid': False,
4     'noupdate': False,
5     'create_uid': (1, 'Administrator'),
6     'create_date': '2026-07-01 08:12:33',
7     'write_uid': (2, 'Ahmed Muhumed'),
8     'write_date': '2026-07-20 14:05:10',
9 }]
```

IMPORTANT METADATA KEYS

- ▶ `id` – database ID
- ▶ `xmlid` – external ID if the record came from XML/data files
- ▶ `noupdate` – whether upgrades should skip updating this data record
- ▶ `create_uid` / `create_date` – who created it and when
- ▶ `write_uid` / `write_date` – who last modified it and when

WHEN TO USE GET_METADATA()

- ▶ Debugging module data and external IDs.
- ▶ Checking whether a record is system/data XML versus user-created.
- ▶ Inspecting create/write audit information quickly.

```
1 for row in self.env['student.student'].browse([15, 16]).get_metadata():  
2     print(row['id'], row['xmlid'], row['write_date'])
```

Method Call

```
1 metadata = self.env['student.student'].browse(15).get_metadata()
```

- ▶ Here, `.get_metadata()` is the method call.
- ▶ Return type reminder: **list of dicts**.

SUMMARY

- ▶ The Odoo ORM lets you work with PostgreSQL through Python recordsets.
- ▶ **create** / **write** / **unlink** / **copy** manage the life cycle of records.
- ▶ **search** / **read** / **search_read** / **search_count** query and fetch data.
- ▶ **name_create** / **name_search** / **default_get** / **get_view** / **fields_get** / **get_metadata** support UI and metadata workflows.
- ▶ **exists** / **ensure_one** / **filtered** / **mapped** / **sorted** / **grouped** help you work safely and clearly with recordsets.

Thank You

Questions?

Ahmed Muhumed

Odoo 17 Development Tutorials